

SAF: A Firewall for Serverless Applications

Pedro Escaleira, Vitor A. Cunha, João P. Barraca, Diogo Gomes, Rui Aguiar
Instituto de Telecomunicações and University of Aveiro, Aveiro, Portugal
{escaleira, vitorcunha, jpbarraca, dgomes, ruilaa}@ua.pt

Abstract—Serverless applications are increasingly implemented as invocation graphs of short-lived functions composed through cloud-managed services, which reduces the coverage offered by perimeter-only firewalls. This article proposes Serverless Application Firewall (SAF), a serverless firewall abstraction that brings Web Application Firewall (WAF)-style, policy-driven request and response filtering to each function boundary by integrating enforcement into provider-managed runtime middleware. Serverless Application Firewall (SAF) aims to limit lateral movement without requiring changes to function code or introducing a centralized enforcement choke point. It further defines a lightweight, function-scoped policy format for expressing these rules. A preliminary open-source Proof of Concept (PoC) demonstrates feasibility, showing that the added latency remains in the millisecond range and grows approximately linearly with the number of per-function firewall rules.

Index Terms—Serverless, Function as a Service (FaaS), Web Application Firewall (WAF), Firewall, Cloud, Cloud Security

I. INTRODUCTION

Serverless platforms have become a popular way to build cloud applications as they remove much of the operational burden of provisioning, scaling, and maintaining runtime infrastructure [1]. This shift also reshapes where controls live: instead of a small set of long-running services behind a perimeter, applications are decomposed into short-lived functions wired together via Cloud Service Provider (CSP)-managed services and event triggers.

This decomposition creates a security tension [2]. CSPs provide mature identity and access management to control which entities can invoke which resources, yet many attacks exploit application semantics rather than identity alone (e.g., malformed inputs, abusive parameter combinations, protocol edge cases, or valid credentials used in unintended ways). In traditional web deployments, Web Application Firewalls (WAFs) help mitigate this by filtering application-layer traffic at ingress (e.g., load balancers or API gateways), a model that is also commonly adopted in serverless.

However, serverless applications rarely behave like a single entry point leading to a trusted interior. They are better described as an invocation graph: functions call services, services trigger functions, and workflows coordinate sequences of steps. Once any node in this graph is reachable, whether exposed publicly, through an internal path, or through a compromised component, it can be used to influence downstream components. A perimeter WAF may reduce risk at the front door, but it does not provide systematic protection for the rest of the application, where later-stage invocations may carry higher privileges and more sensitive effects.

This article proposes the Serverless Application Firewall (SAF) framework, a function-centric alternative to perimeter-only firewalling. Serverless Application Firewall (SAF) adopts

the core spirit of a WAF: a policy-driven inspection and decision-making on application-layer interactions. However, it applies it at each function boundary rather than only at ingress. The main idea is simple: every function invocation and response is an opportunity to enforce developer intent. That intent is expressed through function-scoped policies that define what a function is allowed to receive and emit.

A key design goal of SAF is that enforcement should not depend on a single centralized appliance or gateway. Serverless platforms already interpose provider-managed middleware in the invocation path to handle lifecycle and runtime concerns. SAF leverages that existing interposition point: the same boundary component that mediates function execution can be extended to mediate security decisions, intercepting requests and responses consistently without changing application code paths or forcing all traffic through a single chokepoint. As a result, SAF is distributed by construction and avoids concentrating availability and trust in a singular enforcement tier.

Beyond describing the SAF model, this work demonstrates feasibility with an open-source proof of concept built on Knative, showing how function-boundary firewalling can be integrated into a production-grade serverless platform. The broader goal is to provide a practical and extensible abstraction for firewalling in serverless systems, starting with HTTP-triggered functions, while remaining adaptable to other invocation protocols and internal communication patterns as serverless ecosystems evolve.

This article is organized as follows. Section II reviews the related work on serverless computing and security, as well as WAFs. Then, Section III describes the problem motivating this work, with Section IV following with the SAF framework proposal. Section V describes the Proof of Concept (PoC) implementation on Knative, followed by the preliminary evaluation in Section VI. Finally, Section VII concludes the article and outlines directions for future work.

II. STATE OF THE ART

A. Serverless

Serverless computing is typically presented as a programming and deployment paradigm in which application functionality is realized through a combination of Function as a Service (FaaS) for custom code execution and Backend as a Service (BaaS) for managed capabilities such as storage, messaging, authentication, and API front ends [1]. In this model, the CSP abstracts away most infrastructure-level concerns (provisioning, scaling, and much of the runtime lifecycle) so that developers primarily manage function code and its configuration while relying on managed services for the surrounding platform features.

A central property of serverless applications is their event-driven execution model: they are activated by discrete events, including HTTP requests, message deliveries, database changes, object storage notifications, and scheduled triggers [1]. Consequently, serverless systems are commonly decomposed into many small, purpose-specific functions that interact via managed services and orchestration components rather than through direct, long-lived service-to-service communication. This design enables fine-grained elasticity: individual functions can automatically scale rapidly in response to workload changes and, when idle, scale down to zero, improving resource efficiency and potentially reducing costs. However, simultaneously, this decomposition yields highly distributed applications with many interaction points, increasing the number of security-relevant configurations and trust relationships that must be managed correctly.

Focusing on the function layer, serverless shifts most operational responsibilities away from tenants: developers primarily implement function code, while the CSP provides the build, packaging, deployment, and scaling machinery that integrates this code into the platform. At runtime, functions execute within an isolation boundary, commonly a container or a micro Virtual Machine (microVM) such as Firecracker [3], together with a provider-managed handler (or middleware) that mediates interactions between the platform and the function process. This middleware typically helps orchestrate the function lifecycle (initialization, invocation, and teardown), receives events from the surrounding serverless control plane, and delivers them to the function in a uniform execution model, while also supporting monitoring and operational telemetry. By interposing a stable, provider-controlled runtime layer between the function and underlying infrastructure, the platform decouples application logic from infrastructure details and enables portability of function code across deployments.

B. Serverless Security

The migration to serverless architectures has spurred a growing body of security research, mainly because the shift to small, event-triggered functions alters both the composition and the exposure of cloud applications. Instead of a small set of durable services, functionality is split into many short-lived function executions triggered by external and internal events, and these executions are interwoven with managed services that provide storage, messaging, and orchestration. In such settings, the security boundary is no longer “the application server”: it is a fabric of function invocations and service interactions running atop shared FaaS and BaaS infrastructure, which makes it substantially more challenging to ensure isolation, monitor comprehensively, and reason holistically about the propagation of data and control [2].

The literature repeatedly points to practical failure modes that follow from this distribution and dynamism. Configuration and authorization errors remain common [4], economic abuse, such as Denial-of-Wallet, becomes a realistic threat because resource consumption is directly monetized [5], and establishing reliable end-to-end visibility (e.g., tracking information

flows across ephemeral functions and managed components) is difficult in these environments [6]. Collectively, these findings suggest that serverless security is still addressed through a patchwork of mechanisms rather than a coherent, scalable discipline, leaving significant room for improvement in how protections are specified, enforced, and validated over time [2].

Network security has received increasing attention in serverless security research, reflecting the fact that serverless applications are fundamentally composed of networked interactions among functions, event sources, and managed BaaS services [2]. In particular, the workflow layer, whether explicitly expressed through orchestrators or implicitly through event-chained invocations, has become a primary target for protection mechanisms, as it captures the application’s effective control structure while remaining managed mainly by the CSP. Consequently, a substantial subset of prior work focuses on strengthening the integrity of control- and data-flow across workflow executions and on improving observability so that deviations from intended execution can be detected [6–8]. Despite this progress, to the best of our knowledge, the question of how traditional firewalls, particularly WAF-style application-layer rule systems, can be reinterpreted and enhanced for serverless architectures (including internal invocations, heterogeneous triggers, and provider-managed runtimes) has not yet been studied.

C. Web Application Firewalls

A WAF is an application-layer security mechanism that inspects web traffic, primarily HTTP request and response exchanges, to detect and block malicious patterns before they reach the protected application logic. In contrast to traditional network firewalls that reason mainly over transport and network-layer metadata, WAFs operate on application semantics (e.g., URLs, headers, query parameters, and bodies), enabling mitigation of common web exploits such as SQL injection (SQLi) and Cross-Site Scripting (XSS) through policy-based filtering [9].

Although most currently used WAFs are rule-based [10], the literature has also explored Machine Learning (ML)-assisted and anomaly-based WAF designs to reduce false positives and improve robustness. For instance, Applebaum et al. [11] highlighted that ML can complement rule systems by improving adaptability (including for previously unseen patterns), while noting open issues around evaluation realism and operational constraints. Nguyen et al. [12] later propose combining rule-based WAFs with an auxiliary classifier (based on request language and category cues) to help reduce false positives and support the rule-based decision process, illustrating a representative hybrid direction.

In practice, WAFs are typically deployed as an inline enforcement point placed in front of a web application, commonly via reverse proxies or API gateways that intercept requests and block those deemed malicious according to configured rules [9]. This deployment model is effective for classic (monolithic or three-tier) web applications because most untrusted input enters through a small, stable set of

HTTP ingress endpoints, and the application behind is often composed of comparatively few components. Hence, a single “front door” control point covers a significant fraction of the externally reachable attack surface.

However, serverless applications fundamentally weaken this assumption: beyond external HTTP ingress, substantial application behavior is driven by non-HTTP events (e.g., queue messages, storage notifications, scheduled triggers) and by chains of internal interactions among functions and managed services, so the effective attack surface is distributed across many short-lived executions rather than concentrated behind a single long-lived server boundary. As a result, while conventional WAFs remain helpful in protecting a public API endpoint, they do not, by themselves, yield a *serverless-native* protection model that can express and enforce application-layer policies consistently across heterogeneous triggers and east–west, intra-workflow invocations in a FaaS and BaaS setting. This gap motivates revisiting WAF-style mechanisms for serverless so that enforcement and observability attach to workflows and event propagation paths rather than being limited to a perimeter reverse proxy.

III. MOTIVATION

Serverless applications are commonly decomposed into many independently deployed functions that are composed through workflows, event chains, or service integrations. This composition implies that a significant fraction of application functionality is performed through function-to-function and function-to-service interactions rather than within a single, centralized execution context. Consequently, each function becomes a potential attack target: if any reachable function has a vulnerability, an adversary may compromise it and subsequently leverage its execution context to influence downstream components. In the absence of adequate internal protection mechanisms, a single compromised function can therefore become a pivot point for lateral movement, enabling the attacker to target additional functions and services and to amplify the impact beyond the initially exploited component.

In current cloud practice, application-layer firewalling is typically concentrated at ingress. External traffic is commonly terminated and routed by managed front-door components such as API gateways, reverse proxies, and load balancers, which often integrate WAFs as inline enforcement points. This model is well aligned with traditional web architectures where the application boundary is comparatively stable and can be approximated by a few entry points. However, serverless applications increasingly resemble invocation graphs rather than a single bounded service: even when ingress filtering is present, many security-relevant interactions occur internally, and a compromised internal component may generate requests that never traverse the original ingress choke point.

A similar architectural shift has been explored in the broader cloud-native ecosystem, particularly in microservice architectures where east–west traffic can bypass traditional north–south perimeter controls and thus becomes a key driver of lateral movement. Much of the microservices security literature

and practice therefore emphasizes connectivity governance, restricting which workloads may communicate, through zero-trust micro-segmentation and service-to-service access control. For example, micro-segmentation is often framed as a distributed virtual-firewall model in which fine-grained policies are bound to workloads to reduce lateral movement and restore visibility over internal traffic, and Basta et al. show that constraining internal connectivity can reduce exposure and improve containment after an initial breach [13].

Complementary to this, NIST SP 800-233 [14] characterizes the service mesh as a runtime infrastructure layer in which data-plane proxies mediate application traffic and act as distributed policy enforcement points, including for layer-4 and layer-7 security functions such as authentication and authorization, while also discussing the trade-offs introduced by proxy-based enforcement (e.g., complexity, attack surface, and overhead). However, serverless platforms generally lack an analogous, uniform “per-workload proxy” primitive: because functions are ephemeral and scale rapidly, attaching and operating an always-on sidecar-like enforcement component per function instance is often impractical unless the platform embeds it into the provider-managed runtime itself. This gap motivates moving layer-7, WAF-like checks to the function entry point as a first-class, CSP-managed capability rather than treating it as optional, tenant-operated infrastructure.

These observations motivate the central claim of this article: serverless function interactions should be secured under a zero-trust mindset, in which “internal” origins are not sufficient trust signals. In microservice deployments, comparable goals are commonly pursued through identity-aware service-to-service authorization and traffic governance provided by service meshes and sidecar patterns, with WAFs still often positioned at north–south ingress and other enforcement mechanisms deployed and operated (or at least explicitly configured) at the application or cluster level. In contrast, SAF introduces WAF-like firewalling at the boundary of each serverless function as a platform capability, leveraging the provider-managed runtime middleware to enforce policies transparently. Concretely, when a function receives an invocation, whether from an external client, an orchestrator, or another function, the callee validates and constrains that interaction according to explicit, function-scoped policies. This shifts protection from a perimeter-centric toward a distributed, boundary-centric model aligned with modern cloud-native security goals of limiting lateral movement through internal communication paths.

IV. PROPOSAL

A. Threat Model

In this work, the system model consists of the cloud platform, the tenant application, benign users, and adversaries. The cloud platform encompasses the provider-managed serverless runtime and its associated services, including function sandboxes, runtime middleware, and managed backend services that together implement execution, storage, messaging, and coordination. These platform components and their control planes are treated as trusted: they correctly enforce isolation,

authentication, and authorization, and provide communication channels with integrity and authenticity.

The tenant application is implemented using serverless functions and their configurations, which together define how functions are triggered and how they invoke provider-managed services. Benign users are legitimate entities that invoke the application’s exposed interfaces as intended. They are the primary affected parties, since successful attacks may result in unauthorized actions being taken on their behalf or the compromise of their data.

The adversary is modeled as either (i) an external attacker acting as a client that can invoke public entry points and supply crafted inputs or (ii) a tenant-side insider threat within the application boundary (e.g., a compromised function identity, leaked credentials, or a malicious component deployed in the tenant’s account) that can issue invocations and access resources according to its granted permissions. Within these bounds, the adversary’s objective is lateral movement inside the application: by manipulating inputs, events, and messages, the attacker attempts to reach and misuse other functions, particularly those with access to sensitive state or powerful BaaS operations, to violate the security of benign users while staying within the permissions granted by the existing configuration.

Attacks that require compromising the cloud provider itself (e.g., subverting the control plane, breaking isolation mechanisms, or acting as a malicious insider within the provider organization) are excluded. Likewise, Denial of Service (DoS) or resource-exhaustion attacks against the serverless platform or its networking infrastructure, as well as disruptions caused by global cloud outages, are considered out of scope. The focus is on adversaries who can exploit application-level vulnerabilities and configurations but cannot violate the trust assumptions of the underlying cloud platform.

B. Architectural Overview

Following a zero-trust mentality, Serverless Application Firewall (SAF) is relatively simple: instead of relying on a single firewall at the entry point of a serverless application (Figure 1a), an additional firewall is deployed at each individual function (Figure 1b). This proposal does not replace or weaken the conventional model. Instead, in line with the principle of defense in depth, it introduces an extra layer of protection within the application’s internal topology.

Furthermore, consistent with serverless principles, where the tenant should focus on application business logic while the CSP transparently delivers supporting services, including security services [2], SAF is conceived as part of the middleware already present in serverless sandboxes. As described in Section II-A, serverless platforms package functions with provider-managed middleware that automatically handles the function lifecycle: receiving requests from the platform, converting them to a standard invocation format, and returning the function’s response to the platform for further processing. As shown in Figure 1b, SAF is integrated as an additional, transparent capability of this middleware, deciding whether requests and responses are allowed or denied based on a

priori-defined firewall rules, without requiring changes to the function code or its deployment workflow.

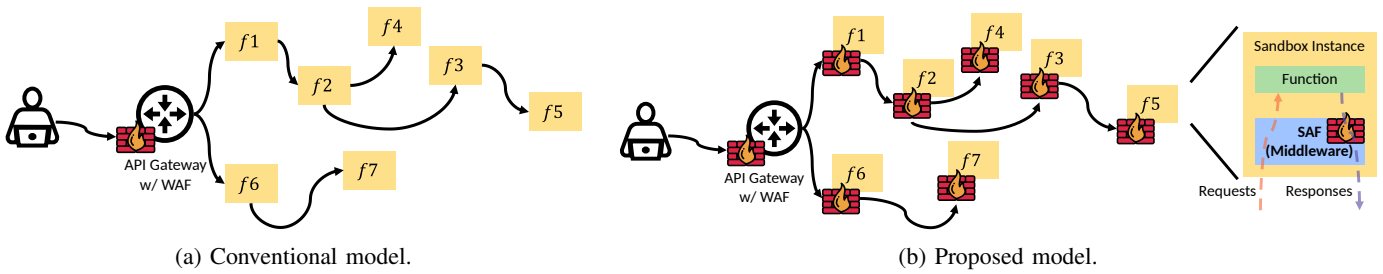
Given how serverless functions are typically executed as small, short-lived processes, their interaction surface with the platform is narrow: they either (i) receive input via standard input (stdin) and emit results via standard output (stdout) through the provider-managed middleware layer or (ii) expose a minimal HTTP server to which that middleware connects. Under either, SAF sits on the same request and response path and can enforce policy before data reaches the function and before results propagate further in the application. Consequently, SAF primarily validates the request and response bodies, since these carry the semantics that most directly affect application behavior and downstream interactions. On platforms where stdin and stdout are the only interfaces between middleware and function, the body effectively becomes the sole feasible inspection target. On platforms using the HTTP-server model, SAF aligns even more closely with a WAF by supporting rules over HTTP-specific elements, such as headers, methods, and status codes, enabling stricter, more expressive filtering at each function boundary.

Accordingly, as with cloud-provided WAFs, SAF policies may be defined either by the CSP (e.g., generic rules that protect functions against standard injection-style payloads) or by the tenant when function-specific controls are required. In both cases, policies are specified *a priori* (i.e., before deployment) and are made available at runtime to the function’s middleware layer. Concretely, when the platform initializes a function instance, the centralized serverless control plane can provision the applicable SAF policy alongside the rest of the function’s execution context, enabling enforcement as soon as the function begins handling requests.

Nevertheless, unlike most WAFs, SAF is inherently decentralized: enforcement is performed locally at each function boundary rather than at a single choke point. Each function instance is equipped with its own firewall, loaded only with the policies that apply to it, which avoids routing internal traffic through a centrally placed enforcement component. This design yields three practical benefits. First, it eliminates a single point of failure and a potential performance bottleneck on the application’s critical path. Second, compromise or malfunction of one SAF instance is contained to that function and does not directly prevent other functions from executing and enforcing their policies. Third, policy evaluation becomes lighter-weight and more scalable because each firewall processes only a small, function-scoped ruleset instead of an application-wide policy corpus. In short, SAF turns firewalling from a perimeter control into a per-function, least-privilege control that scales with the serverless execution model.

V. IMPLEMENTATION

To assess the feasibility of our proposal and obtain preliminary performance results, we developed an open-source prototype of SAF [15] on top of Knative [16]. In Knative Serving, the request path to a function instance is mediated by the queue-proxy (Knative’s function middleware), which



```

firewall:
  request:
    default-action: accept
    rules:
      # 1. Drop if method is not POST
      - action: drop
        expression: |
          .REQUEST.METHOD
          | ascii_upcase
          | . != "POST"
      # 2. Template/code injection detection (simple example)
      - action: drop
        expression: |
          ({.REQUEST.BODY.username?, .REQUEST.BODY.password?}
          | map(tostring) | join(" "))
          | test("{}{|}}|{%|-%}|__class__|__mro__")
  response:
    default-action: accept
    rules:
      # 1. No stack traces or internal errors (simple example)
      - action: drop
        expression: |
          .RESPONSE.BODY.error?
          | type == "string"
          | and test("(?(i)(stacktrace|exception|traceback|sql)")

```

forwards invocations using HTTP requests and responses. Therefore, our prototype enforces policies over HTTP transactions at the function boundary: in addition to inspecting request and response bodies, it can match on HTTP-specific fields such as methods, headers, or status codes. The implementation itself is divided into two main components: (i) the `rules reference`, which defines the policy model and rule syntax, and (ii) the `firewall enforcer`, which loads these policies at runtime and applies them to each invocation.

The rules reference defines a lightweight “policy format” consistent with standard WAF rule systems (ordered rules and a default action). Following common cloud configuration practices, policies can be serialized as YAML or JSON. Listing 1 shows an example SAF policy that specifies separate rule sets for requests and responses. Each policy includes a default action (accept, reject, or drop) and an ordered list of rules evaluated sequentially. Each rule is expressed via an expression field written in jq [17], enabling concise matching across structured payloads and HTTP metadata. At deployment time, policies are attached to each function via the Knative Service resource’s metadata and thus travel with the service definition managed by the control plane.

As for the firewall enforcer, it is implemented as an extension to Knative’s `queue-proxy`, thereby augmenting the existing per-function middleware with SAF enforcement capabilities. On startup, the SAF enforcer retrieves the firewall

policy associated with the function, validates it, and compiles it into internal data structures suitable for efficient evaluation during request processing. During regular operation, the `queue-proxy` continues to receive inbound requests and forward them to the user function, returning the function's response to the platform. The difference is that SAF now evaluates the configured rules on both the request and response paths. For each request and response, the corresponding rule list is evaluated sequentially in the order defined by the developer. When a rule matches (i.e., its expression evaluates to true), the firewall immediately applies that rule's action. If, on the other hand, no rule matches, SAF applies the request or response policy's `default-action` to decide whether that request or response is allowed or denied.

VI. EVALUATION

Building on the evaluation approach of Datta et al. [7], our assessment focuses on operational expenses that impact the practical adoption of inline policy enforcement in serverless environments: container image size, function deployment and teardown times, and end-to-end latency. This methodology is a good fit for SAF because, like Valve [7], SAF enforces security policies on the invocation path and may therefore introduce overhead at both provisioning time (image distribution and initialization) and at request-processing time (per-invocation checks). To support reproducibility, all scripts and experimental results are released as open-source artifacts [15].

In contrast to approaches like Valve, which instrument or modify the function container image, our SAF prototype for Knative integrates the enforcement logic into the `queue-proxy` sidecar rather than into the user function image. Consequently, any increase in image size is confined to the sidecar, leaving the function build artifact unchanged. This separation is relevant in practice because container images are layered, so updates to a shared sidecar image can be amortized across many services and revisions. In contrast, changes to each function image would require rebuilding and redistributing it per function.

A. SAF Performance

1) *Methodology*: All experiments were performed on a Kubernetes cluster running K3s v1.32.6 and Knative v1.19. The cluster comprised three nodes (one controller and two worker nodes), each deployed as a Virtual Machine (VM) with 16 vCPUs and 16 GB of RAM. Workload generation and experiment orchestration were executed from a separate

VM on the same network, which issued deployment requests, monitored application lifecycle events, and collected execution measurements. This benchmarking host was provisioned with 4 vCPUs and 8 GB of RAM. All VMs were provisioned and managed in a shared OpenStack-based private cloud.

2) *Build size*: Using Knative `queue-proxy` v1.20.1, we compared the container image size of the baseline `queue-proxy` with our version extended with the Enforcer. The original image is 46.2MB, whereas the extended image is 70.7MB, corresponding to an increase of roughly 1.53 $\times$. However, this overhead is not incurred by each protected function separately. Because `queue-proxy` is deployed as a sidecar container, multiple Pods on the same node can reuse the same read-only image layers, amortizing storage and download costs across functions that share the same node.

3) *Deployment and Teardown*: To isolate the overhead introduced by SAF, these experiments used a minimal Python function that handles a POST request and returns the instance hostname. This choice minimizes application-side processing cost so that measured differences primarily reflect the firewall behavior rather than the function logic itself. Then, we evaluated two scenarios: (i) a baseline deployment using the unmodified Knative `queue-proxy` and (ii) a SAF-protected deployment in which the `queue-proxy` includes our extension. For each run, we measured the function deployment time, and once the service became ready (for SAF, after all rules were loaded and compiled into the required internal structures), we measured the termination (teardown) time. Each scenario was executed 550 times to reduce noise and improve statistical confidence. To study how policy complexity affects lifecycle overheads, the SAF scenario was also repeated with different numbers of rules. This is particularly relevant at deployment time, as it captures the delay until a function is ready to accept requests and thus directly impacts elasticity and cold-start behavior in serverless environments.

The results are shown in Figure 2, which plots, for each ruleset size, the difference between the time observed with SAF enabled and the baseline time. Overall, the deployment time gap (Figure 2a) grows with the number of rules, reaching approximately $2.29 \pm 1.52s$ at 500 rules. This implies that a function protected by SAF takes longer to become ready to serve requests than the same function without SAF. Although a configuration with 500 rules is extreme for a typical serverless function (which is usually small and self-contained), the observed overhead highlights an important area for future work. In particular, subsequent iterations of SAF should investigate more efficient rule-loading and compilation strategies to reduce their contribution to cold-start latency.

Regarding teardown, the results summarized in Figure 2b were somewhat unexpected. Intuitively, one might expect teardown time to increase with the number of rules, since larger rule sets imply larger internal data structures and, therefore, more state to release when the firewall process terminates. However, the measurements do not show a monotonic increasing trend: the difference decreases from $-0.05 \pm 1.30s$ at 0 rules to $-0.70 \pm 1.30s$ at 350 rules, before increasing again

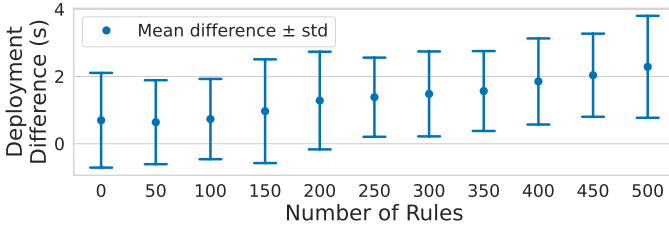
to $-0.37 \pm 1.31s$ at 500 rules.

A plausible explanation is that GoLang, the programming language used to develop the firewall, triggers the garbage collector when the heap reaches a specific size. Under this hypothesis, smaller rule sets may not trigger garbage collection (or may trigger it less often), leaving more memory to be reclaimed during shutdown, whereas larger rule sets may trigger garbage collection earlier, reducing memory left to be reclaimed at teardown, decreasing the observed teardown time. Past a certain threshold, additional allocations may again not be cleared by garbage collection immediately, producing the observed pattern. Consequently, we conducted additional experiments (results available in the repository [15]) in which garbage collection was explicitly triggered immediately after deployment and heap usage was monitored. Under these conditions, the heap size measured before the manual garbage collection step showed a clearer dependence on the number of rules, decreasing up to a threshold and then increasing again. Moreover, teardown time grew with the number of rules, matching the expected behavior when shutdown cleanup scales with the amount of allocated state.

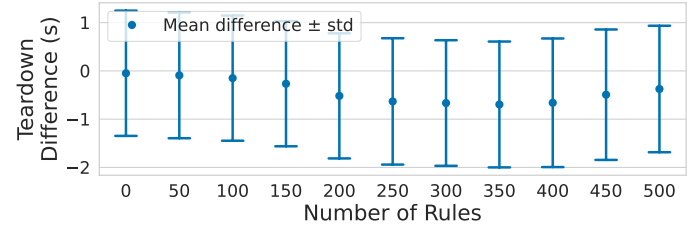
4) *Latency*: Using the same minimal function as in Section VI-A3, as well as the same two scenarios, we quantify SAF’s latency overhead while keeping application-side processing noise low. For each run, we issued a request to the function and recorded the end-to-end request latency, repeating the measurement 550 times per scenario. To assess the impact of policy complexity, we repeat the SAF measurements while varying the number of configured rules, thereby characterizing how ruleset size influences per-request latency.

The obtained results are presented in Figure 3. For each configuration, we report the latency difference relative to the baseline (unmodified `queue-proxy`), i.e., baseline latency subtracted from the latency observed with SAF. Across the tested range, the added latency increases approximately linearly with the number of rules, suggesting that per-request enforcement cost is dominated by rule evaluation. With zero rules, SAF introduces no meaningful penalty, with a measured difference of $0.18 \pm 1.10ms$. At 500 rules, the overhead rises to $56.01 \pm 4.92ms$. While this increase is non-negligible, it remains in the millisecond range and corresponds to an intentionally stress-tested configuration: as serverless functions are small and purpose-specific, their SAF policies are expected to be similarly narrow and context-dependent. For example, a function that does not interact with a database is unlikely to require extensive SQLi filtering, whereas an input-validation function might prioritize stricter payload constraints.

Moreover, even when we consider the typical number of rules used by well-known CSPs, they remain in this order of magnitude. At the time of writing, Oracle Cloud Infrastructure’s WAF documentation notes that its policy contains approximately 680 managed rules [18]. This figure illustrates how cloud-provider WAFs typically ship broad, perimeter-oriented rule catalogs intended to cover a wide range of application types. In contrast, SAF can rely on substantially smaller, function-scoped subsets tailored to each function’s narrow



(a) Deployment time difference.



(b) Teardown time difference.

Fig. 2: Number of rules impact on deployment and teardown times.

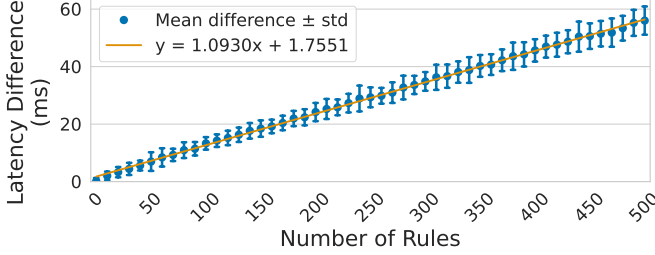


Fig. 3: Number of rules impact on function latency.

interface and expected inputs and outputs. Consequently, CSPs adopting SAF could expose smaller, curated policy subsets (or reusable templates) tailored to typical serverless function roles (e.g., input validation, data access, message handling, or file processing) so that each function enforces only the rules relevant to its specific interface and expected behavior. This preserves the benefits of provider-managed expertise while keeping per-function rule sets compact, reducing evaluation overhead, and simplifying policy management for tenants.

Nevertheless, this is a preliminary PoC validate the proposal and obtain initial performance insights, requiring further engineering to reach production maturity. In realistic serverless deployments, functions are commonly composed into workflows or event-driven architectures, where a single user request may trigger multiple functions in sequence. In such cases, the per-function overhead of SAF can accumulate along the execution path, becoming more noticeable. For this reason, production-ready implementations of SAF should incorporate performance optimizations. For example, WAF implementations can benefit from techniques such as memoization of repeated decisions and pre-filtering to eliminate irrelevant rules early. Cloudflare reports achieving a 40% performance improvement in its WAF using these and other approaches [19].

VII. CONCLUSION

This article introduced SAF, a serverless-native firewall abstraction that applies WAF-style, policy-driven filtering at each function boundary by integrating enforcement into provider-managed runtime middleware. By distributing enforcement across functions rather than concentrating it at a single ingress choke point, SAF aims to reduce opportunities for lateral movement in serverless invocation graphs while remaining transparent to functions.

A PoC implementation on Knative demonstrates feasibility by extending the `queue-proxy` sidecar to load per-function policies and enforce them on HTTP requests and responses.

The evaluation shows that overhead increases with ruleset size, particularly during startup (policy loading) and per-request processing, highlighting the need for engineering optimizations in future work. Beyond its immediate firewalling use case, SAF points to a broader direction for serverless security: leveraging the CSP-managed function middleware as a standardized, per-function enforcement point where additional security services can be composed [2, 6, 7]. Future work includes optimizing the current implementation, studying ML-assisted generation of function-tailored policies, and extending the model and evaluation to capture richer rule complexity and additional serverless triggers and communication patterns

REFERENCES

- [1] Z. Li et al., “The Serverless Computing Survey: A Technical Primer for Design Architecture,” *ACM Computing Surveys*, vol. 54, no. 10s, pp. 1–34, Jan. 2022.
- [2] P. Escalera et al., “A systematic review on security mechanisms for serverless computing,” *Cluster Computing*, vol. 28, no. 7, 2025.
- [3] A. Agache et al., “Firecracker: Lightweight virtualization for serverless applications,” *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020*, pp. 419–434, 2020.
- [4] I. Polinsky et al., “GRASP: Hardening Serverless Applications through Graph Reachability Analysis of Security Policies,” *WWW 2024 - Proceedings of the ACM Web Conference*, pp. 1644–1655, 2024.
- [5] D. Kelly et al., “Denial of wallet—Defining a looming threat to serverless computing,” *Journal of Information Security and Applications*, vol. 60, 2021.
- [6] D. S. Jegan et al., “Guarding Serverless Applications with Kalium,” *32nd USENIX Security Symposium, USENIX Security 2023*, vol. 6, pp. 4087–4104, 2023.
- [7] P. Datta et al., “Valve: Securing Function Workflows on Serverless Computing Platforms,” *The Web Conference 2020 - Proceedings of the World Wide Web Conference, WWW 2020*, pp. 939–950, 2020.
- [8] P. Datta et al., “ALASTOR: Reconstructing the Provenance of Serverless Intrusions,” in *Proceedings of the 31st USENIX Security Symposium, Security 2022*, 2022, pp. 2443–2460.
- [9] R. A. Muzaki et al., “Improving Security of Web-Based Application Using ModSecurity and Reverse Proxy in Web Application Firewall,” *2020 International Workshop on Big Data and Information Security, IWBIS 2020*, pp. 85–90, 2020.
- [10] A. MK et al., “An empirical study on the evaluation and enhancement of OWASP CRS (Core Rule Set) in ModSecurity,” *Computers and Security*, vol. 160, 2026.
- [11] S. Applebaum et al., “Signature-based and Machine-Learning-based Web Application Firewalls: A Short Survey,” *Procedia CIRP*, vol. 189, pp. 359–367, 2021.
- [12] T. C. H. Nguyen et al., “Improving Web Application Firewalls with Automatic Language Detection,” *SN Computer Science*, vol. 3, no. 6, 2022.
- [13] N. Basta et al., “Towards a zero-trust micro-segmentation network security strategy: An evaluation framework,” in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, 2022, pp. 1–7.

- [14] R. Chandramouli et al., “Service Mesh Proxy Models for Cloud-Native Applications,” National Institute of Standards and Technology, Tech. Rep., Oct. 2024.
- [15] P. Escaleira, *SAF: Serverless Application Firewall*, Available: <https://tinyurl.com/iscc-saf>, 2026.
- [16] “Knative,” Knative Project. [Online]. Available: <https://knative.dev/>.
- [17] “jq,” jqlang. [Online]. Available: <https://jqlang.org/>.
- [18] “WAF Protection Rules,” Oracle Cloud Infrastructure. [Online]. Available: <https://docs.oracle.com/en-us/iaas/Content/WAF/Tasks/wafprotectionrules.htm>.
- [19] “Making the WAF 40% faster,” Cloudflare. [Online]. Available: <https://blog.cloudflare.com/making-the-waf-40-faster/>.